arXiv:2602.15829v1 [cs.LG] 17 Feb 2026

Operationalising the Superficial Alignment Hypothesis via Task Complexity

Tomás Vergara-Browne [1 2] **Darshan Patil** [1 3] **Ivan Titov** [4 5] **Siva Reddy** [1 2 6]
Tiago Pimentel [* 7] **Marius Mosbach** [* 1 2]

Abstract

The superficial alignment hypothesis (SAH) posits that large language models learn most of their knowledge during pre-training, and that post-training merely surfaces this knowledge.[1] The SAH, however, lacks a precise definition, which has led to (i) different and seemingly orthogonal arguments supporting it, and (ii) important critiques to it. We propose a new metric called **task complexity**: the length of the shortest program that achieves a target performance on a task. In this framework, the SAH simply claims that pre-trained models drastically reduce the complexity of achieving high performance on many tasks. Our definition unifies prior arguments supporting the SAH, interpreting them as different strategies to find such short programs. Experimentally, we estimate the task complexity of mathematical reasoning, machine translation, and instruction following; we then show that these complexities can be remarkably low when conditioned on a pre-trained model. Further, we find that pre-training enables access to strong performances on our tasks, but it can require programs of gigabytes of length to access them. Post-training, on the other hand, collapses the complexity of reaching this same performance by several orders of magnitude. Overall, our results highlight that task adaptation often requires surprisingly little information—often just a few kilobytes.

* Equal advising. [1]Mila Quebec AI Institute [2]McGill University [3]Université de Montréal [4]University of Edinburgh [5]University of Amsterdam [6]Canada CIFAR AI Chair [7]ETH Zürich. Correspondence to: Tomás Vergara-Browne <tomas.vergarabrowne@mila.quebec>, Tiago Pimentel <tiago.pimentel@inf.ethz.ch>, Marius Mosbach <marius.mosbach@mila.quebec>.

Preprint. February 18, 2026.

[1]To avoid confusion with the AI safety literature, we will say models are 'adapted', rather than 'aligned', to a task. We use the name 'superficial alignment hypothesis', however, as opposed to 'superficial adaptation hypothesis', due to historical reasons.

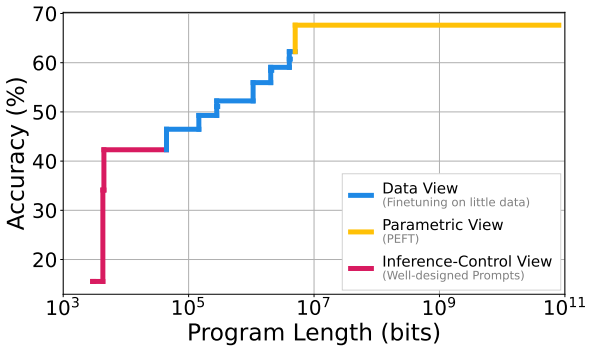

Figure 1. Pareto curve of program length vs. performance for Olmo3-7B on GSM8K. We argue that prior works about the superficial alignment hypothesis can be seen as proposing different approaches to find short programs to solve a task, and we find that these different views inform different regions of this Pareto curve.

1. Introduction

The superficial alignment hypothesis (SAH) states that a large language model's *"knowledge and capabilities are learned almost entirely during pre-training, while alignment teaches it which subdistribution of formats should be used when interacting with users"* (Zhou et al., 2023a). This hypothesis has been supported from different angles, e.g., we can adapt pre-trained models to solve complex tasks by: 1 finetuning it on few examples (Zhou et al., 2023a; Ye et al., 2025), 2 updating only a few of its parameters (Han et al., 2024; Li & Kim, 2024), or 3 using well-designed prompts (Lin et al., 2024; Lake et al., 2025).

Despite its appeal, however, the SAH suffers from imprecise definitions of important terms such as 'knowledge', 'capabilities', and 'subdistribution of formats'. This ambiguity has created two main problems. First, arguments supporting the hypothesis are seemingly orthogonal: some emphasise the small data requirements to adapt a model (1 above), others the number of parameters needed to be updated (2), and still others the fact that adaptation can be achieved through inference-time control alone (3). This makes it unclear whether these reflect a common underlying phenomenon. Second, critics have exploited these undefined terms to challenge the hypothesis. Raghavendra et al. (2024) and Lambert (2025), for instance, interpret that if a model contains 'knowledge' and 'capabilities' for a task, its performance on the task should saturate very easily; by

showing that, on many tasks, performance saturation requires a significant amount of fine-tuning or post-training, they conclude the SAH is incomplete. These tensions highlight the need for a more precise operationalisation of the SAH that can bring clarity to this debate.

In this paper, we provide a clearer definition of the superficial alignment hypothesis, grounded in algorithmic information theory. We first define the **complexity of a task at performance δ** as the length of the shortest program to achieve performance δ on it. We then argue that the SAH can be operationalised as claiming that many complex tasks—i.e., tasks for which no short program exists which achieves a certain target performance on it—become low-complexity once we give our programs access to a pre-trained model. This perspective places seemingly orthogonal arguments supporting the SAH on a common, quantitative footing: arguments based on data efficiency, parameter efficiency, and inference control define different classes of programs whose lengths can be directly compared (see Figure 1). These arguments are, in fact, instances of the same underlying principle: there exist short programs to achieve target performance in the task. The arguments do not compete with each other, but rather simply use different strategies to find such programs.

Experimentally, we measure the task complexity of three diverse NLP tasks (mathematical reasoning, machine translation and instruction following) when given access to the weights of either: SmolLM3 3B (Bakouch et al., 2025), Olmo3 7B, or Olmo3 32B (Olmo et al., 2025). We find that programs as small as 1.2×10^6 bits (which is 151 kilobytes, or the size of a single image from ImageNet; Russakovsky et al., 2015) can adapt these large language models (LLMs) to achieve strong performance on some of these tasks.

We further analyse how task complexity evolves during training by examining randomly initialised, pre-trained, and post-trained checkpoints of SmolLM3 and Olmo3 7B. Strong performance on these tasks first becomes accessible with pre-trained checkpoints, but requires the use of long programs to plateau the obtained performance (typically on the order of megabytes to gigabytes). Post-trained checkpoints enable access to the same strong performance, however, with dramatically shorter programs.

Overall, our work provides a rigorous framework to understand the superficiality of LLM adaptation from an algorithmic information perspective: adapting a pre-trained model for a task often requires just kilobytes of information.

2. Perspectives on Superficial Adaptation

We present 3 different views from prior work which highlight the idea that knowledge about a task is already present in pre-trained models, and not learned at later adaptation stages. These views are: the data view, the parametric view, and the inference-control view.

① **Data view.** *Adaptation is superficial because we can finetune an LLM on few data points to adapt it to a new task.*

Adapting a pre-trained LLM to a task requires remarkably little data in comparison to the amount of data used in pre-training (Zhou et al., 2023a; Qi et al., 2024; Ye et al., 2025; Muennighoff et al., 2025). For instance, Zhou et al. (2023a) show that 1000 examples are enough to adapt a pre-trained LLM into a strong instruction-following model. When adopting the data view, the contrast between the small amount of data necessary for adaptation and the much larger amounts of data used in pre-training can be interpreted as evidence for the superficiality of adaptation.

② **Parametric view.** *Adaptation is superficial because only few parameters need to be changed to adapt an LLM to a new task.*

There is ample evidence showing that fine-tuning a pre-trained LLM requires a surprisingly small amount of trainable parameters (Li & Liang, 2021; Hu et al., 2022; Liu et al., 2022a; Ben Zaken et al., 2022; Liu et al., 2022b; Dettmers et al., 2023; Yadav et al., 2025; Han et al., 2024; Li & Kim, 2024; Ping et al., 2024; Liu et al., 2024; Huang et al., 2025; Morris et al., 2026). For example, LoRA (Hu et al., 2022) allows us to efficiently finetune LLMs by adding trainable adapters which represent a small fraction (less than 1%) of a model's parameters. The contrast between the small amount of trainable parameters during adaptation and the large amount of parameters needed in pre-training can be seen as evidence for the superficiality of adaptation.

③ **Inference-control view.** *Adaptation is superficial because we can keep all LLM weights frozen, and only make simple changes to its inference code to adapt it to a task.*

In some cases, it is not even necessary to modify the parameters of a pre-trained LLM at all to perform well on a downstream task (Brown et al., 2020; Lin et al., 2024; Hewitt et al., 2024; Minder et al., 2025; Braun et al., 2025; Chen et al., 2025; Karan & Du, 2025). For example, Lin et al. (2024) show how to carefully design a prompt which makes a pre-trained LLM follow instructions. Under this view, the fact that no weights have to be changed at all can be seen as evidence for the superficiality of adaptation.

3. The Superficial Alignment Hypothesis

While the previous three views may seem orthogonal, their results can still be interpreted as evidence of superficiality. Why is that? We argue here that this is because they all evoke a sense that most of the information required to perform well on a task is already present in the pre-trained model. In this section, we introduce definitions based in algorithmic information theory (Li & Vitányi, 2008) that al-

low us to formalise this intuition, unifying these three views in the process. We start by defining the concept of a task.

Definition 3.1. *Let a* ***task*** $\mathtt{T}$ *be a tuple* $(\mathcal{X}, \mathcal{Y}, p, \mathcal{S})$*, where:*

- $\mathcal{X}$ *and* $\mathcal{Y}$ *are, respectively, an input and output space.*
- $p : \mathcal{X} \to [0, 1]$ *is a probability distribution over inputs* $x \in \mathcal{X}$*.*
- $\mathcal{S} : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$ *is a scoring function which evaluates the quality of an input-output pair.*

Let us use the example of solving grade school math problems as our task $\mathtt{T}$. Both $\mathcal{X}$ and $\mathcal{Y}$ are the space of text strings, and p is a distribution over grade school math problems written in text. For example, p would assign a non zero probability to an $x \in \mathcal{X}$ such as *"how many fruits are three apples and two oranges?"*. The scoring function $\mathcal{S}$ takes a problem x and an answer y, and returns 1 if y correctly solves x, and 0 otherwise. So given a $y \in \mathcal{Y}$ of *"five"*, we would have $\mathcal{S}(x, y) = 1$.

Now, let M be a Universal Turing Machine. This machine takes in a program $\mathsf{P} \in \mathcal{P}$ and an input $x \in \mathcal{X}$, to compute an output $y \in \mathcal{Y}$.[2] We can write $\mathrm{M}(\mathsf{P}, x)$ as the output of running program P on input x (if it halts). As throughout our discussion we keep M unspecified and fixed, we will use $\mathsf{P}(x)$ to represent $\mathrm{M}(\mathsf{P}, x)$. We can now define the score of a program P on a task $\mathtt{T}$ as its expected performance on it.

Definition 3.2. *Given a task* $\mathtt{T} = (\mathcal{X}, \mathcal{Y}, p, \mathcal{S})$ *and a program* P*, we define the* ***score*** *of* P *on* $\mathtt{T}$ *as:*

$$\mathtt{score}_{\mathtt{T}}(\mathsf{P}) \stackrel{\text{def}}{=} \mathbb{E}_{x \sim p}[\mathcal{S}(x, \mathsf{P}(x))] \tag{1}$$

Coming back to our example, we can design a program P that achieves some score on task $\mathtt{T}$. The program P may include any logic necessary to output some response y whenever an input x is provided. The program must, however, be completely self-contained, meaning that it cannot access anything that is not directly encoded in P or x. The score of this program P is represented as $\mathtt{score}_{\mathtt{T}}(\mathsf{P})$, and we will say that program P *solves* $\mathtt{T}_\delta$ if $\mathtt{score}_{\mathtt{T}}(\mathsf{P}) \geq \delta$.

3.1. Task Complexity

We now characterise the complexity of a task $\mathtt{T}$ at performance δ as the length of the shortest program that solves it.

Definition 3.3. *The* ***complexity of task*** $\mathtt{T}$ ***at performance*** $\boldsymbol{\delta}$ *is defined as:*

$$\mathrm{C}(\mathtt{T}_\delta) \stackrel{\text{def}}{=} \min_{\mathsf{P} \in \mathcal{P}} \{\mathit{len}(\mathsf{P}) : \mathtt{score}_{\mathtt{T}}(\mathsf{P}) \geq \delta\} \tag{2}$$

[2]We define a program as a string of bits, and we assume that the elements in $\mathcal{X}$ or $\mathcal{Y}$ can be encoded as bit-strings, i.e., $\mathcal{P} = \{0, 1\}^*$ and $\mathcal{X}, \mathcal{Y} \subseteq \{0, 1\}^*$, where * denotes the Kleene star operator.

This metric quantifies the amount of information required to solve (or represent) task $\mathtt{T}$ at a certain performance level, being tightly related to both Kolmogorov complexity and rate-distortion theory (as further discussed in Section 3.3). If a $\mathtt{T}_\delta$ is less complex than another $\tilde{\mathtt{T}}_\delta$, then we can solve $\mathtt{T}_\delta$ with a shorter program than what we need to solve $\tilde{\mathtt{T}}_\delta$.

In our running example, $\mathrm{C}(\mathtt{T}_{90\%})$ would represent the length of the minimum program to solve grade school math problems with 90% accuracy. Presumably, this program might be very long, as it requires understanding natural language, planning the steps for which to solve the problem and then computing the answer. Given access to an LLM, however, even such a complex task might be greatly simplified. This motivates our definition of conditional task complexity: the length of the minimum program P which, given access to a model's weights θ (represented as a bit-string[3]), achieves a certain performance at task $\mathtt{T}$. We will use P_θ to represent a program P with access to a model's weights θ, where we treat this access to θ as an additional input of the program, i.e., $\mathsf{P}_\theta(x) = \mathsf{P}(x, \theta)$.

Definition 3.4. *The* ***complexity of task*** $\mathtt{T}$ ***at performance*** $\boldsymbol{\delta}$ ***conditioned on*** $\boldsymbol{\theta}$ *is defined as:*

$$\mathrm{C}(\mathtt{T}_\delta \mid \theta) \stackrel{\text{def}}{=} \min_{\mathsf{P} \in \mathcal{P}} \{\mathit{len}(\mathsf{P}) : \mathtt{score}_{\mathtt{T}}(\mathsf{P}_\theta) \geq \delta\} \tag{3}$$

While the task of solving grade school math problems with 90% accuracy is very complex, i.e., $\mathrm{C}(\mathtt{T}_{90\%})$ is presumably very high, access to a pre-trained model θ may lead to a considerably lower complexity $\mathrm{C}(\mathtt{T}_{90\%} \mid \theta)$. This reduction in complexity defines the amount of information that a model θ contains about a task $\mathtt{T}$ at performance δ.

Definition 3.5. *The* ***information a model*** $\boldsymbol{\theta}$ ***contains about a task*** $\mathtt{T}$ ***at performance*** δ *is defined as:*

$$\mathrm{I}(\mathtt{T}_\delta; \theta) \stackrel{\text{def}}{=} \mathrm{C}(\mathtt{T}_\delta) - \mathrm{C}(\mathtt{T}_\delta \mid \theta) \tag{4}$$

We derive a few theoretical properties of these proposed metrics in Appendix A, leveraging existing results in algorithmic information theory for related concepts.

3.2. Adaptability and SAH

So far, our definitions focused on tasks and their complexity. The SAH, however, is about pre-trained models, discussing how adaptable they are to new tasks. We thus now put forward a definition of how adaptable a model is to a task. Specifically, we say that a model is (b, δ)-adaptable for a task $\mathtt{T}$ if there exists a program shorter than b which allows the model to solve $\mathtt{T}_\delta$.

[3]We additionally consider the tokenizer for a language model to be part of the bit-string θ.